COMS4040A & COMS7045A Assignment 1 – Part 1

Hand-out date: 14:00, March 16, 2026
Due date: 23:59 pm, April 1, 2026

Note: Assignment 1 will be handed out in two parts, Part 1 and Part 2, where Part 1 is on CUDA C/C++ programming and Part 2 will be on OpenMP programming.

Objectives

1. Design and implement parallel algorithms using OpenMP and CUDA C/C++ with a focus on achieving scalable performance.
2. Evaluate the performance of a parallel algorithm implementation.
3. Writing a short scientific report.

Instructions

1. This is an individual assignment.
2. The due date is strictly enforced. There is no late penalty until **8:00 AM on April 2**. After that, **10% of your total mark will be deducted for each 24-hour period (or part thereof)**. No submissions will be accepted after **11:59 PM on April 3**, and failure to submit by then will result in a mark of **zero**.
3. Refer to the section **Hand-ins** for submission details.
4. The total mark for this assignment (Part 1) is **42**.

Problems

Problem 1. Image Convolution Using CUDA C/C++ [34]

Introduction

Convolution is an array operation in which each output element is computed as a weighted sum of neighboring input elements. The weights used in the weighted sum calculation are defined by an input mask array, referred to as the convolution mask here. The same convolution mask is typically used for all elements of the array. Convolution is commonly used in various forms in signal processing, digital recording, image processing,

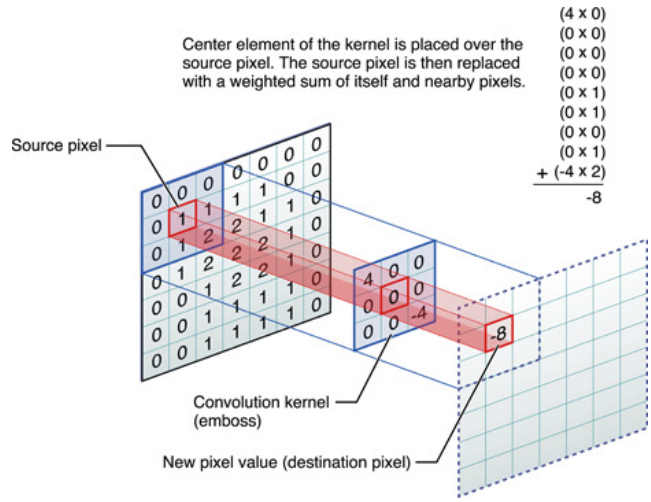

Figure 1: Performing convolution operation

video processing, and computer vision. In these application areas, convolution is often performed as a filter that transforms signals and pixels into more desirable values.

Convolution typically involves a significant number of arithmetic operations on each data element. For large data sets such as high-definition images and videos, the amount of computation can be very large. Each output element can be calculated independently of each other, making it well-suited for parallel computing. On the other hand, there is a substantial amount of input data sharing for computing neighboring output data elements with non-trivial boundary handling. This makes convolution an important use case of sophisticated tiling methods and input data staging methods.

When applied in image processing tasks, convolution leads to results such as noise removal, edge detection and sharpening of details. If an image is represented as a 2D discrete signal $Y \in \mathbb{Z}^{M \times N}$, we can perform the (discrete) convolution in 2-dimension using a kernel $F \in \mathbb{Z}^{L \times P}$ as

$$(Y * F)[i,j] = \sum_{l=-a}^{a} \sum_{p=-b}^{b} Y[i+l, j+p] F[l+a, p+b], \quad 0 \le i < M,\ 0 \le j < N, \tag{1}$$

where $a = \lfloor L/2 \rfloor, b = \lfloor P/2 \rfloor$ for a kernel size of $L \times P$.

The convolution operation in Equation (1) is actually a scalar product between the corresponding filter weights and the pixel values within a window centered at each pixel location, where the window size is determined by the dimensions of the filter. Figure 1 illustrates the convolution using a small 3×3 kernel. The design of the convolution filter requires careful selection of the weights to achieve desired results.

Implementation Considerations

Image convolution can be efficiently implemented on massively parallel hardware, since the same operator gets executed independently for each image pixel. A naive implementation would simply execute the convolution for each pixel by one CUDA thread, read all values inside the filter area from the global memory, and write the result. (Note that this approach is inefficient.)

In this assignment, you are going to implement image convolution using the following methods.

1. Serial computation [10/34]
2. CUDA implementation using global memory [12/34]
3. CUDA implementation using shared memory [12/34]

In your implementation:

1. Use the filters in Table 1 for testing the convolution result.
2. A `pgm` test image is given. To load a `pgm` image or write a `pgm` image, you may use the relevant CUDA SDK functions. A CUDA SDK sample program is provided.
3. The pixel values outside the image boundaries should be treated as 0 values.
4. Your code should be written in a way that the size of convolution mask is arbitrary in the case of **averaging** filter, that is, you should test it on different sizes of convolution masks, e.g., 3 by 3, 5 by 5, and 7 by 7.
5. When applicable, describe any further relevant implementation design choices in your report.

Sharpen	Emboss	Average
$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 9 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 & -1 \end{bmatrix}$	$\frac{1}{25}\begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$

Table 1: Convolution masks

Hand-Ins

1. Source code (well-commented, including sequential and both parallel versions). [4]
2. Script or instructions to compile and run the code. [4]
3. Output data file.
4. Report with performance analysis. A combined report for Problems 1 & 2 named as `<yourStudentNo>_report.pdf`: You may organize the contents in two sections, one for each problem. In each section: [34[1]]
   - Description of your implementation and experiments

[1] This mark is a combination of the marks indicated in Problems 1 & 2.

- Performance results (tables/graphs of classification accuracies, execution times and speedup).
- Discussion and analysis of results.
- Challenges faced and solutions.
- Cite the sources where applicable.
- **Note:** Do not put your results in a table or graph without giving any discussions or explanations of those in the text.
- A conclusion for each section.

5. Submit the report and source codes separately via the respective submission link provided.
6. Submit your report to Turnitin to produce a similarity check report. [Negative marks[2]]
7. The source code submission should be a single compressed file named as `<yourStudentNo>_assignment1.tar.gz`. The compressed file should be extracted to a folder named `<yourStudentNo>_assignment1`. In this folder, organize the codes for different problems in separate folders.
8. For each problem, include `Makefiles` to build your code, run script(s), such as `run.sh`, to run your programs, and readme file with instructions to compile and run your code.
9. The report should be written using latex. Template tex files are provided in this regard. You just need to edit `assignment1_report.tex` file and compile `template.tex` using latex.

[2]Depending on the similarity score, varying negative marks will be given. A similarity score below 15% is not penalized.